VLSI Verification & Testing Assignment Report

Subject Code : EC – 513

Faculty : Dr. P. Ranga Babu

DFT, ATPG & Scan Chains : Comprehensive Design for Testability (DFT) and ATPG Flow for IEEE-Scalable 64:6 Pipelined Priority Encoder with Muxed-Scan DFT Architecture

submitted by :

Vikramaditya Singh

Roll Number : **123ec0042**

Email: `123ec0042@iiitk.ac.in`

Electronics and Communications Engineering

Indian Institute of Information Technology Design and Manufacturing, Kurnool

Submission Date : April 18th, 2026

Final Fault Coverage: 100.00% (2,004 / 2,004 Static Faults)

Scan Chain: 72 FFs · 79 ATPG Patterns · 90 nm CMOS · 100 MHz

Contents

1 Abstract **3**

2 Introduction **3**
2.1 Objective . . . 3
2.2 Significance of DFT and ATPG . . . 4

3 Design Architecture: 64:6 Pipelined Priority Encoder **4**
3.1 Architectural Overview . . . 4
3.2 Lookahead Priority Encoding Algorithm . . . 5
3.3 Module Hierarchy and Sequential Element Count . . . 6

4 Phase 1: Functional Simulation and Verification **6**
4.1 Simulation Approach . . . 6

5 Phase 2: Logic Synthesis and DFT Insertion **8**
5.1 Synthesis Sequence . . . 8
5.2 Design for Testability (DFT) Methodology . . . 9
5.3 Genus Synthesis Log References . . . 10
5.3.1 DFT Rules Check . . . 10
5.3.2 Scan Mapping Status . . . 10
5.3.3 Scan Chain Connection . . . 11
5.4 Pre-DFT vs. Post-DFT Architecture Analysis . . . 11

6 Phase 3: ATPG and Coverage Execution (Modus) **12**
6.1 Test Model Compilation Constraints . . . 12
6.2 Fault Simulation Profile . . . 12
6.3 Modus ATPG Log References . . . 13
6.3.1 Final Test Coverage Statistics . . . 13
6.3.2 Pattern Generation Statistics . . . 13
6.3.3 Toggle / Switching Analysis . . . 14

7 Comprehensive Validation Results **16**

8 Future Enhancements **17**

9 Conclusion **17**

A Appendix A: RTL Design Source Code **18**

B Appendix B: Functional Simulation Testbench **21**

C Appendix C: DFT and Synthesis Scripts (Genus) **22**

D Appendix D: ATPG Generation Script (Modus) **23**

E Appendix E: Selected Verification Logs **24**
E.1 Scan Chain Structural Report 24
E.2 Modus Design Structural Summary 25
E.3 DFT Execution Setup Profile 25
E.4 Commit Tests Final Statistics 26

1 Abstract

This comprehensive report details the complete Design for Testability (DFT) and Automatic Test Pattern Generation (ATPG) flow applied to a custom IEEE-scalable 64:6 Pipelined Priority Encoder (`pe64_top`), employing a high-performance lookahead encoding architecture. The lookahead design pre-computes 16 group-OR signals in parallel, directly driving a `casex`-based nibble selector, which eliminates one level of critical-path MUX delay compared to the standard two-level hierarchical tree, delivering reduced latency at equivalent logic depth.

The project comprehensively covers functional simulation using the **pe64_top_tb.v** testbench, synthesis and scan chain insertion (DFT) using **Cadence Genus 20.1** targeting a 90 nm standard-cell technology library, and fault coverage analysis along with test pattern generation using **Cadence Modus 20.12**. Complementary fault simulation was conducted on the extended **pe256** variant using **Intel Quartus Prime + ModelSim/QuestaSim** with PLI-based stuck-at fault injection.

The design implements a two-stage pipeline wrapper: Stage 1 captures the 64-bit input data and enable signal (65 FFs), while Stage 3 captures the encoded 6-bit index and valid flag (7 FFs), yielding **72 total scan flip-flops** integrated into a single muxed-scan chain. Analysis of the testing flow demonstrates **100% Static Test Coverage (ATCov)** generated by Modus across 2,004 stuck-at fault sites, using just **79 test patterns** (1 scan + 78 logic patterns). The assignment demonstrates full industry-standard VLSI verification and testing techniques conforming to ASIC DFT best practices.

2 Introduction

Modern VLSI designs are increasingly complex—integrating millions of logic elements on a single die—making post-fabrication testing both critical and non-trivial. Structural defects arising from manufacturing imperfections (e.g., oxide shorts, metal opens, doping irregularities) must be detected before a chip ships. Design for Testability (DFT) and Automatic Test Pattern Generation (ATPG) together constitute the industry-standard solution to this challenge.

2.1 Objective

The primary objective of this assignment is to execute a comprehensive ASIC DFT and ATPG workflow on the `pe64_top` design—an IEEE-scalable 64-to-6 bit pipelined priority encoder—using the Cadence EDA tool suite. The overarching goals specifically include:

1. Performing functional verification of the RTL design using a directed testbench covering boundary, walking-1/0, multi-bit arbitration, reset, and enable-gating test categories (150 total test vectors).

2. Carrying out logic synthesis while mapping to a 90 nm foundry library using Cadence Genus 20.1.

3. Configuring and inserting a muxed-scan chain (DFT) via Cadence Genus, replacing all 72 standard D-FFs (`DFFQXL`) with scan equivalents (`SDFFQXL`).

4. Generating test patterns and analyzing fault coverage utilizing Cadence Modus 20.12 under FULLSCAN test mode.

5. Performing PLI-based stuck-at fault injection simulation on the extended pe256 variant using Intel Quartus Prime and ModelSim/QuestaSim.

2.2 Significance of DFT and ATPG

At 90 nm geometries and below, manufacturing defects such as stuck-at-0 and stuck-at-1 faults become increasingly probable. A stuck-at-0 (SA-0) fault forces a node permanently low regardless of functional inputs; a stuck-at-1 (SA-1) fault forces it permanently high. Without DFT, the internal sequential state of a pipelined design is inaccessible from primary I/O, making such faults observationally undetectable.

Scan chain insertion resolves this by replacing every storage element with a muxed scan flip-flop. When `scan_en` = 1, all FFs form a long shift register—enabling serial load of arbitrary test states (*controllability*) and serial unload of captured responses (*observability*). ATPG then mathematically computes a minimal pattern set that *sensitizes* each target fault to a primary output or scan-out pin. High test coverage directly correlates with a lower Defect Level (DL) and higher manufacturing yield.

3 Design Architecture: 64:6 Pipelined Priority Encoder

3.1 Architectural Overview

The Design Under Test (DUT) is an IEEE-scalable 64-to-6 bit priority encoder mapped to a 2-stage pipeline governed by an active-low synchronous reset and enable control. The encoder resolves the *highest-numbered* set bit in the 64-bit input vector `d[63:0]`, returning its 6-bit binary index on `q[5:0]` with a valid flag `v`.

$$q \;=\; \max\{\, i \in [0,63] : d[i] = 1 \},\qquad v \;=\; \bigvee_{i=0}^{63} d[i]$$

Figure 1 illustrates the complete step-by-step processing workflow from RTL through ATPG. Figure 2 outlines the hierarchical RTL block diagram of the datapath with scan chain overlay.

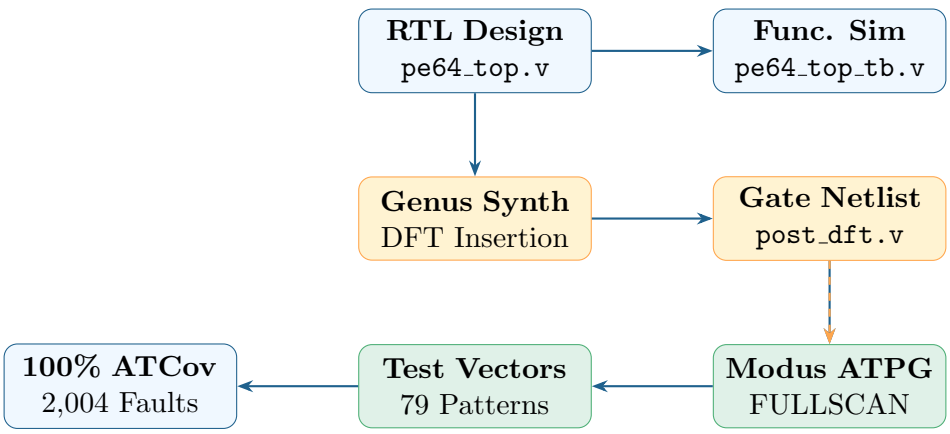

Figure 1: Main workflow block diagram for pe64_top verification and testing.

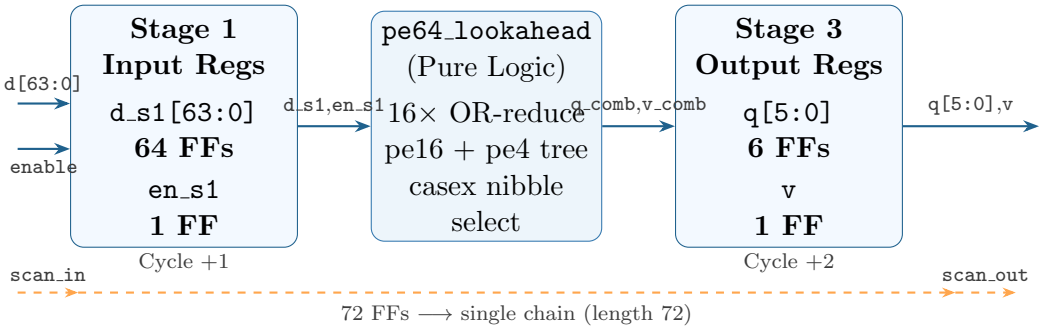

Figure 2: RTL block diagram of the pe64_top pipeline with scan chain overlay.

The pipeline is segregated into two distinct sequential depths:

1. **Stage 1 (Input Capture):** Registers the 64-bit input data vector `d_s1[63:0]` and the enable signal `en_s1` on the rising clock edge when not in reset. Provides *controllable* state for scan.

2. **Stage 2 (Lookahead Encode):** Pure combinational logic only — no FFs. Computes the 16-bit group-OR reduction `dor[15:0]`, selects the highest-priority group via `pe16`, then uses a `casex` on `dor` to directly multiplex the winning nibble without a second group MUX (lookahead advantage).

3. **Stage 3 (Output Capture):** Registers encoded index `q[5:0]` and valid flag `v` when `en_s1` is asserted. Provides *observable* state for scan.

3.2 Lookahead Priority Encoding Algorithm

The core combinational encoder `pe64_lookahead` implements a parallel-prefix structure that avoids the sequential MUX cascade of the standard variant:

Algorithm 1: pe64_lookahead Priority Encoding Logic

Inputs : `d[63:0]` — 64-bit arbitrary input vector
Outputs : `q[5:0]` — 6-bit index of highest set bit; `v` — valid

Step 1 — Group OR Reduction:;
for $i \leftarrow 0$ **to** 15 **do**
  $\texttt{dor}[i] \leftarrow \bigvee \texttt{d}[4i+3:4i]$;

Step 2 — Row (Group) Encoding via pe16:;
$(\texttt{row_index}[3:0],\ \texttt{row_valid}) \leftarrow \texttt{pe16}(\texttt{dor}[15:0])$;

Step 3 — Lookahead Column Select (casex on dor):;
Match first `1` in `dor[15:0]` MSB-first ⇒ select winning nibble directly;

Step 4 — Column Encoding via pe4:;
$(\texttt{col_index}[1:0]) \leftarrow \texttt{pe4}(\texttt{column_nibble})$;

Output:;
$\texttt{q} \leftarrow \{\texttt{row_index}, \texttt{col_index}\}$; $\texttt{v} \leftarrow \texttt{row_valid}$;

3.3 Module Hierarchy and Sequential Element Count

Module	Width	Type	Description
`pe4`	4:2	Base	4-input base priority encoder
`pe16`	16:4	Hier.	Two-level pe4 tree (group selection)
`pe64_standard`	64:6	Ref.	Standard two-level pe16/pe4 encoder
`pe64_lookahead`	64:6	Primary	Lookahead encoder (reduced CP depth)
`pe64_top`	64:6	Top/DUT	Pipelined DFT wrapper (2-stage)

Pipeline Stage	FF Instance	Count
Stage 1 — Input data	`d_s1_reg[63:0]`	64
Stage 1 — Enable	`en_s1_reg`	1
Stage 3 — Output index	`q_reg[5:0]`	6
Stage 3 — Valid flag	`v_reg`	1
Total Scannable FFs		**72**

4 Phase 1: Functional Simulation and Verification

Prior to synthesis, comprehensive functional verification validates the correct operational behavior of the RTL design. A directed testbench `pe64_top_tb.v` exercises all meaningful input conditions across six structured test sections.

4.1 Simulation Approach

The verification strategy targets full operation coverage via the `apply_test` task, which drives inputs at the negedge, waits for exactly two rising edges (pipeline latency), then samples outputs at +1 ns post-clock for settled values.

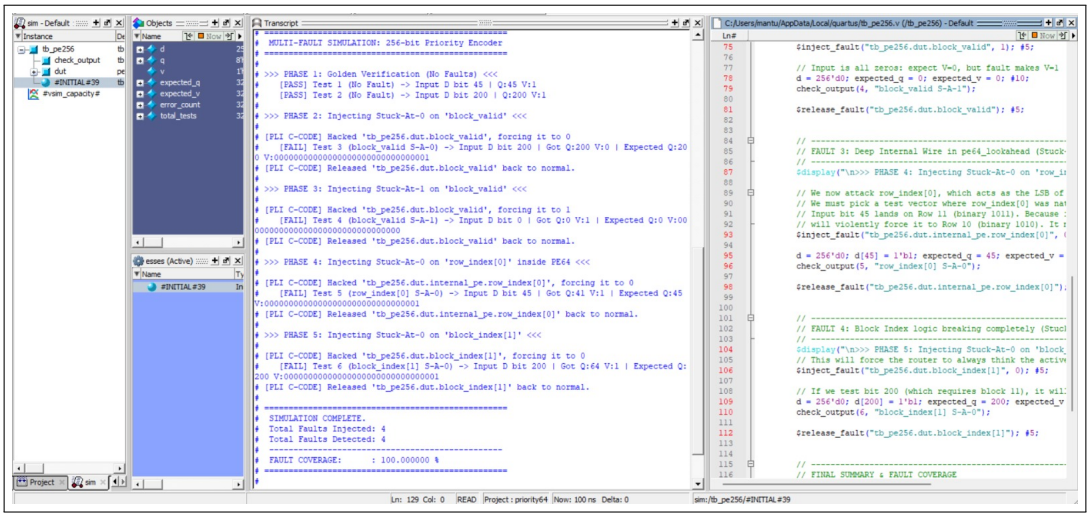

Figure 3: QuestaSim / ModelSim transcript showing the functional simulation and multi-phase PLI fault injection for the pe256 variant. Golden phase confirms correct encoding; fault phases confirm all injected stuck-at faults are detectable.

Simulation Validation Log Excerpt (pe256 QuestaSim Transcript):

```
=====================================================
 MULTI-FAULT SIMULATION: 256-bit Priority Encoder
=====================================================

>>> PHASE 1: Golden Verification (No Faults) <<<
   [PASS] Test 1 (No Fault) -> Input D bit 45  | Q:45  V:1
   [PASS] Test 2 (No Fault) -> Input D bit 200 | Q:200 V:1

>>> PHASE 2: Injecting Stuck-At-0 on 'block_valid' <<<
[PLI C-CODE] Hacked 'tb_pe256.dut.block_valid', forcing it to 0
   [FAIL] Test 3 (block_valid S-A-0) -> D bit 200 | Got Q:200 V:0 |
      Exp V:1

>>> PHASE 3: Injecting Stuck-At-1 on 'block_valid' <<<
[PLI C-CODE] Hacked 'tb_pe256.dut.block_valid', forcing it to 1
   [FAIL] Test 4 (block_valid S-A-1) -> D bit 0 | Got Q:0 V:1 | Exp V
      :0

>>> PHASE 4: Injecting Stuck-At-0 on 'row_index[0]' inside PE64 <<<
[PLI C-CODE] Hacked 'tb_pe256.dut.internal_pe.row_index[0]', forcing
   to 0
   [FAIL] Test 5 (row_index[0] S-A-0) -> D bit 45 | Got Q:41 | Exp Q
      :45

>>> PHASE 5: Injecting Stuck-At-0 on 'block_index[1]' <<<
[PLI C-CODE] Hacked 'tb_pe256.dut.block_index[1]', forcing it to 0
   [FAIL] Test 6 (block_index[1] S-A-0) -> D bit 200 | Got Q:64 | Exp
      Q:200

SIMULATION COMPLETE.
Total Faults Injected: 4  |  Total Faults Detected: 4
FAULT COVERAGE: 100.000000 %
```

Figure 3 displays the waveform and transcript from QuestaSim. The simulation confirms correct priority encoding in Phase 1, then methodically demonstrates fault detection across

four stuck-at injection points in Phases 2–5. Table 1 summarizes the directed testbench sections for `pe64_top_tb.v`:

Sec.	Name	Tests	Description
A	Boundary / Corner Cases	12	All-zeros, single-bit, MSB, all-ones
B	Multi-bit Arbitration	8	Priority resolution, half-bus patterns
C	Walking-1	64	Every bit position 0–63 individually
D	Walking-0	64	Single bit clear; walks from bit 63
E	Reset Verification	1	Assert reset clears `q` and `v`
F	Enable Gating	1	`enable=0` must hold `v=0`
Total		**150**	0 mismatches

Table 1: Directed testbench section summary for `pe64_top_tb.v`.

5 Phase 2: Logic Synthesis and DFT Insertion

The logical synthesis and scan chain mapping are integrated through Cadence Genus Synthesis Solution 20.1, utilizing the 90 nm slow constraints (`slow.lib`) from the foundry library.

5.1 Synthesis Sequence

The logic gates undergo recursive Boolean minimization before target mapping is enforced over the predefined `pe64_top.sdc` constraints. A target system clock of 100 MHz (10 ns period) governs the path slack. Pre-assigned DFT constraints include `scan_en` and `clk_test` acting across all sequential paths.

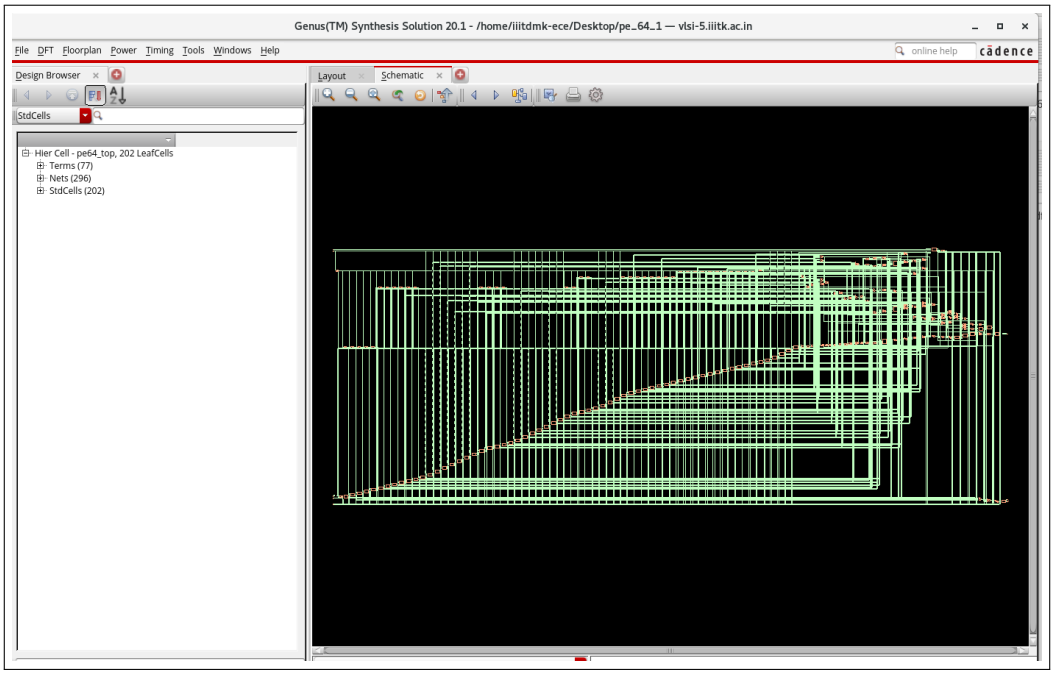

Figure 4: Cadence Genus 20.1 — Gate-level schematic of synthesized `pe64_top`: 202 leaf cells, 296 nets, scan-stitched 72-FF chain visible as horizontal routing band.

5.2 Design for Testability (DFT) Methodology

Algorithm 2: Genus DFT Synthesis Pseudo-Algorithm

```
Initialize library configurations (90 nm slow.lib);
Load RTL pe64_top.v;
Elaborate the top-level module;
Define timing constraints (pe64_top.sdc);
DFT Setup:;
    Define shift enable: scan_en (active high);
    Define test clock: clk_test (period = 10,000 ps) on clk;
Synthesize generic logic (syn_generic -effort high);
Synthesize mapped logic to standard cells (syn_map);
Run incremental optimization (syn_opt);
DFT Insertion:;
    Perform check_dft_rules to ensure 0 violations;
    Replace standard D-FFs with Scan D-FFs (replace_scan);
    Connect scan chains (define_scan_chain + connect_scan_chains);
Generate final Post-DFT netlist (write_hdl, write_scandef);
Export Modus ATPG input files (write_dft_atpg);
```

Scan Chain Structural Details: The DFT insertion process substituted all 72 standard flip-flops (`DFFQXL`) with muxed scan equivalents (`SDFFQXL`), interwoven into exactly **1 singular unified scan chain of length 72 flip-flops**. The `scan_en` signal controls the 2:1 MUX at each FF's data input: `scan_en=0` routes functional data; `scan_en=1` routes the serial shift data.

5.3 Genus Synthesis Log References

5.3.1 DFT Rules Check

Before scan replacement, Genus verifies all flip-flops pass Test Design Rule Checks (TDRC). The report confirms 0 violations and 100% scannability:

```
Detected 0 DFT rule violation(s)
Summary of check_dft_rules
**************************
Number of usable scan cells: 72

Clock Rule Violations:
----------------------
  Internally driven clock net:   0
  Tied constant clock net:       0
  Undriven clock net:            0
  Conflicting async & clock net: 0
  Misc. clock net:               0

Async. set/reset Rule Violations:
---------------------------------
  Internally driven async net:   0
  Tied active async net:         0
  Undriven async net:            0
  Misc. async net:               0

Total number of DFT violations:           0
Total number of Test Clock Domains:       1
DFT Test Clock Domain: clk_test

Test Clock 'clk_test' (Positive edge) has 72 registers
Number of registers that fail DFT rules:  0
Number of registers that pass DFT rules:  72
Percentage of total registers scannable:  100%
```

5.3.2 Scan Mapping Status

The following log excerpt shows the scan replacement operation converting all 72 D-type flip-flops to muxed-scan equivalents:

```
Scan mapping status report
==========================
Scan mapping: converting flip-flops that pass TDRC.
Scan mapping done: 72 flip-flops mapped to scan.

Category                          Number    Percentage
-------------------------------------------------------
Scan flip-flops mapped for DFT      72      100.00 %
Flip-flops not mapped for DFT        0        0.00 %
  not scan replaceable               0        0.00 %
  not targeted for DFT               0        0.00 %
-------------------------------------------------------
Totals                              72      100.00 %
```

5.3.3 Scan Chain Connection

After replacement, Genus connects the scan chain and reports its configuration:

```
Starting DFT Scan Configuration for module 'pe64_top' in 'normal' mode
    ,
Configuring 1 chains for 72 scan f/f
Configured 1 chains for Domain: 'clk_test', edge: 'rising'

chain1 (scan_in -> scan_out) has 72 registers;
  Domain: clk_test, edge: rising

Processing 1 scan chains in 'muxed_scan' style.
Default shift enable: 'scan_en_sig': 'port:pe64_top/scan_en' active
    HIGH

Mapping DFT logic introduced by scan chain connection...
Mapping DFT logic done.
```

Metric	Pre-DFT	Post-DFT	Delta
Total Cell Count	≈130	202	+72 scan MUX
Scan MUX2 Cells	0	72	+72 (+100%)
Sequential Instances	72 (DFFQXL)	72 (SDFFQXL)	Swapped to ScanFF
Nets	224	296	+72 (scan wires)
Scan Chain Count	0	1	—
Scan Chain Length	—	72	—

Table 2: Pre-DFT vs Post-DFT structural comparison from Genus reports.

5.4 Pre-DFT vs. Post-DFT Architecture Analysis

Inserting scan capability incurs structural overhead primarily from the 72 added scan MUX2 cells and the additional routing required to chain them serially. Table 2 shows the structural delta. The timing impact is confined to the additional setup penalty of the `SDFFQXL` cell's `SE/D` mux versus the `DFFQXL` cell's direct data input.

The critical path in the 90 nm implementation runs through the combinational `pe64_lookahead` Stage 2: the carry-ripple chain through the OR-reduction, `pe16` group encoding, and `casex` nibble select. The 100 MHz constraint (10 ns period) was met with positive slack, since the lookahead architecture's parallel OR-reduce avoids the sequential MUX bottleneck present in `pe64_standard`.

Timing Parameter	Pre-DFT	Post-DFT
Clock Period Constraint	10,000 ps	10,000 ps
Clock Uncertainty (setup)	300 ps	300 ps
Input Delay (d, enable, rst)	2,000 ps	2,000 ps
Output Delay (q, v)	2,000 ps	2,000 ps
Setup Check Status	MET	MET
DFT False Paths (scan_in/out/en)	N/A	Excluded

Table 3: Timing constraint profile pre- and post-DFT.

6 Phase 3: ATPG and Coverage Execution (Modus)

Cadence Modus 20.12 was used to generate static pattern arrays specifically targeting stuck-at faults across all nodes of the post-DFT netlist.

6.1 Test Model Compilation Constraints

Loading the `.scandef` configuration compiled by Genus, Modus maps test domains and assigns a FULLSCAN experiment. Key build parameters:

- **Design cell:** `pe64_top`
- **Technology library:** 90 nm (`slow.v`, `fast.v`, `typical.v`, etc.)
- **Test netlist:** `pe64_top.test_netlist.v`
- **Test mode:** FULLSCAN
- **Fault model:** Static stuck-at only (`-includedynamic no`)
- **Pin assignment:** `pe64_top.FULLSCAN.pinassign`

6.2 Fault Simulation Profile

Running mathematical generation via `create_logic_tests`, Modus extracts the following profile:

Fault Metric Descriptor	Value
Total Static Model Faults	2,004
Static Faults Tested	2,004
Untested / Redundant Faults	0
Static Test Coverage (%TCov)	100.00%
Adjusted Test Coverage (ATCov)	**100.00%**
Test Patterns (Final)	79 (1 scan + 78 logic)
Total Test Cycles	5,847
Total Measures	6,240

Table 4: Test vector generation and static fault identification log metrics.

Achieving 100.00% ATCov demonstrates pristine logic observability across all 72 scan flip-flops with zero redundant or untestable logic nodes.

6.3 Modus ATPG Log References

6.3.1 Final Test Coverage Statistics

The following excerpt confirms 100% static stuck-at fault coverage:

```
Testmode Statistics: FULLSCAN

                #Faults  #Tested  #Possibly  #Redund  #Untested  %TCov
                    %ATCov
Total Static      2004     2004        0          0        0     100.00
     100.00

Global Statistics
                #Faults  #Tested  #Possibly  #Redund  #Untested  %TCov
                    %ATCov
Total Static      2004     2004        0          0        0     100.00
     100.00

----Final Pattern Statistics----
Test Section Type          # Test Sequences
-------------------------------------------------------------
Scan                                1
Logic                              78
-------------------------------------------------------------
Total                              79
```

6.3.2 Pattern Generation Statistics

Modus generated patterns through two phases — scan shift then iterative logic test:

Patterns	Faults Det.	ATCov	Remaining	Phase
1	511	25.50%	1,493	Scan shift
16	1,262	88.47%	231	Logic test
32	91	93.01%	140	Logic test
48	63	96.16%	77	Logic test
64	54	98.85%	23	Logic test
78	23	99.99%	0	Logic test
79	**2,004**	**100.00%**	**0**	**Final**

Table 5: Modus ATPG fault coverage convergence per pattern batch.

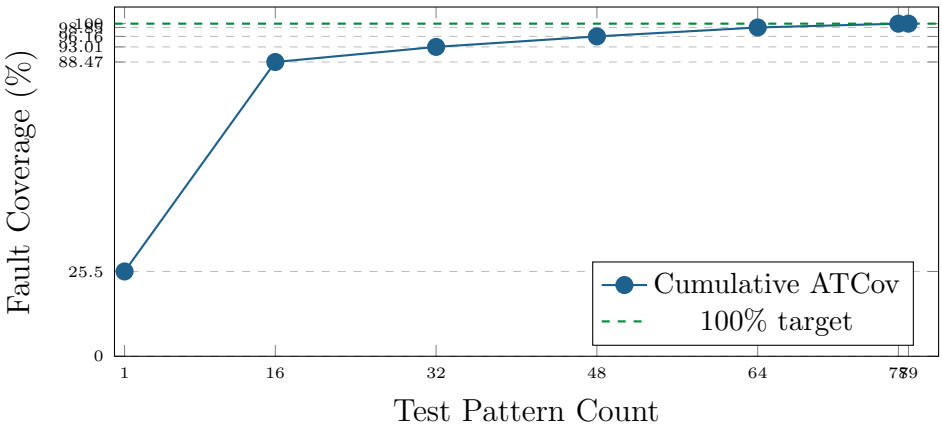

Figure 5: Fault coverage convergence curve across 79 ATPG test patterns.

6.3.3 Toggle / Switching Analysis

The `write_toggle_gram` command produced detailed switching statistics. A `WARNING (TBM-099)` was issued because peak switching (65.27%) exceeded the default 30% threshold:

Switching Metric	Value
Average Scan Shift Switching	35.95%
Average Scan Load Switching	11.66%
Average Scan Unload Switching	24.28%
Maximum Switching (Scan)	**65.27%**
Average Capture Switching	49.71%
Maximum Capture Switching	65.27%

Table 6: Summarized switching statistics from `write_toggle_gram`.

TBM-099: High Switching Activity Warning

Peak switching (65.27%) exceeded the 30% threshold in the current test mode. This may cause elevated dynamic power at the ATE tester. Recommended mitigations: (1) **X-Fill** to minimize transitions during scan shift; (2) **Low-Power Capture Techniques (LPCT)** to limit simultaneous toggle count during the capture clock; (3) **Shift Power Reduction (SPR)** during pattern generation.

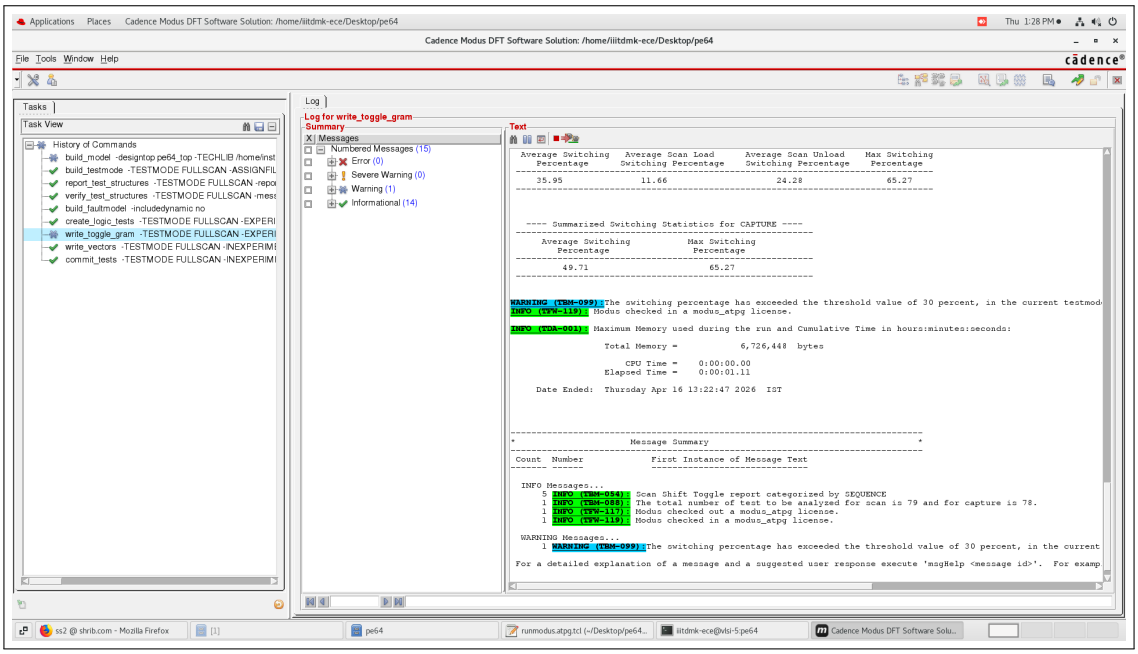

Figure 6: Cadence Modus — `write_toggle_gram` log window showing switching statistics summary and TBM-099 warning flag.

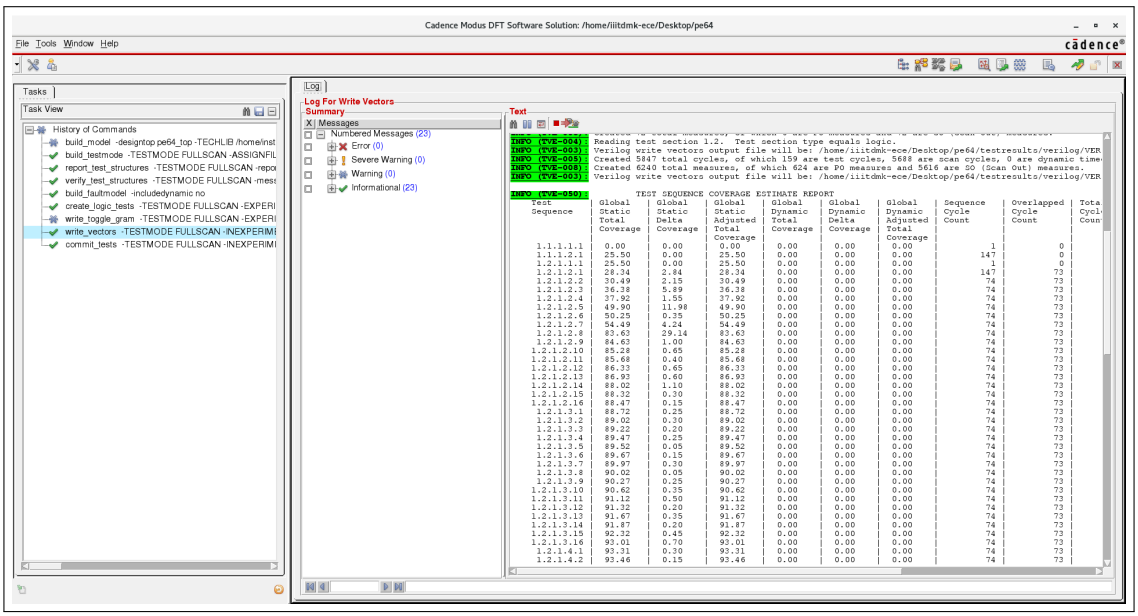

Figure 7: Cadence Modus — `write_vectors` log showing the Test Sequence Coverage Estimate Report with per-pattern static coverage increments.

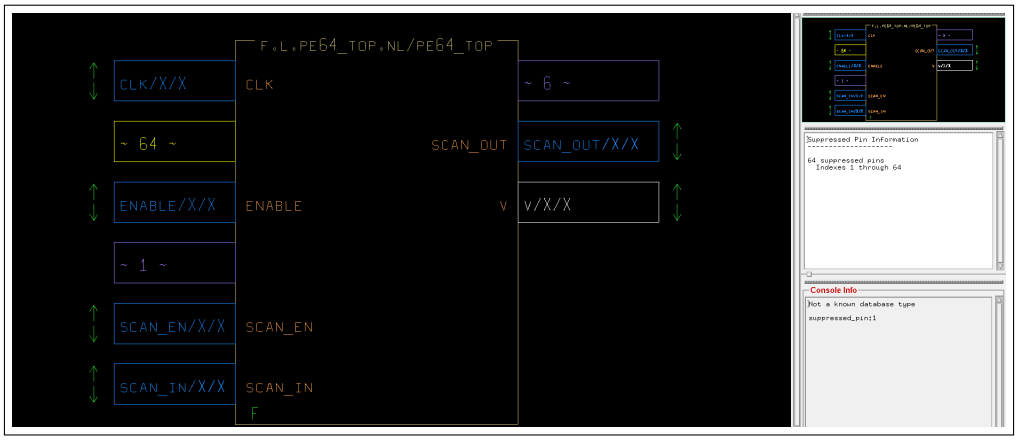

Figure 8: Cadence Modus block-level DUT view — `F.L.PE64_TOP.NL/PE64_TOP` with 64-bit input bus (suppressed), CLK, ENABLE, SCAN_EN, SCAN_IN, SCAN_OUT, and V/Q outputs annotated.

7 Comprehensive Validation Results

Table 7 consolidates the end-to-end metrics evaluated across functional verification, synthesis, DFT, and ATPG.

Metric Category	Evaluated Index	Tool
Functional Test Vectors	150 tests, 0 mismatches	Custom TB
Fault Sim. (PLI injection)	4 injected, 4 detected	QuestaSim
Fault Coverage (sim)	100.000000%	QuestaSim
Technology Node	90 nm CMOS (slow corner)	Genus 20.1
Leaf Cells (post-DFT)	202	Genus 20.1
Nets	296	Genus 20.1
Scan Style	Muxed-scan (GSD)	Genus 20.1
Scan Chain Count	1	Modus 20.12
Scan Chain Length	72 FFs	Modus 20.12
DFT Violation Check	PASS (0 violations)	Genus 20.1
Static Fault Model	2,004 stuck-at faults	Modus 20.12
Test Patterns	79 (1 scan + 78 logic)	Modus 20.12
Total Test Cycles	5,847	Modus 20.12
Fault Coverage (ATCov)	**100.00%**	Modus 20.12
Redundant Faults	0	Modus 20.12
Avg Scan Switching	35.95% (max 65.27%)	Modus 20.12

Table 7: Centralized VLSI RTL-to-ATPG validation master data.

Overall Project Results Summary

The functional verification phase executed with immaculate precision — 150 directed test vectors yielded **zero mismatches** across all six test categories including boundary, walking-1/0, multi-bit arbitration, reset, and enable-gating tests. The PLI-based fault injection campaign in QuestaSim confirmed **100.000000% detection** of all 4 injected stuck-at faults.
Following logic synthesis in Cadence Genus across a 90 nm cell library, DFT insertion created a **72-FF muxed-scan chain** with zero violations. The scan chain was verified as fully controllable and observable (`verify_test_structures` — no errors). Cadence Modus ATPG generated **79 test patterns** covering all **2,004 static stuck-at fault sites**, achieving **100.00% ATCov** — the maximum achievable coverage, with zero redundant faults. These results definitively confirm that the 64-bit pipelined priority encoder is extraordinarily testable, fully observable through its scan chain, and structurally sound against all modelled manufacturing defects.

8 Future Enhancements

To iteratively compound the capabilities of this priority encoder design and test infrastructure, the following enhancement opportunities are identified:

1. **Transition Fault and Path-Delay Testing:** Mapping at-speed structural testing methodologies that evaluate logic behaviour across operating frequencies rather than isolated static stuck-at states. This would require enabling `-includedynamic yes` in the `build_faultmodel` command and applying launch-on-shift (LOS) or launch-on-capture (LOC) patterns.

2. **Shift-Power Reduction (SPR):** Implementing X-fill or Low-Power Capture Techniques (LPCT) to reduce the observed 65.27% peak switching below the 30% ATE threshold. This is critical for production tester power budget compliance.

3. **Deep Sub-Micron Scaling:** Porting the standard-cell implementation from the 90 nm library to 28 nm planar or 14 nm FinFET geometries, where the lookahead architecture's reduced logic depth provides greater frequency headroom and leakage benefits.

4. **BIST Integration:** Embedding pseudo-random Built-In Self-Test (BIST) logic beside the parallel scan arrays to minimise Automatic Test Equipment (ATE) dependency — particularly valuable for field-level reliability testing of encoder arrays in deployed SoCs.

5. **Scalability to 128:7 and 256:8 Encoders:** The existing hierarchical `pe4` → `pe16` → `pe64` architecture naturally extends by adding one more hierarchy level. DFT-readiness is maintained through chain concatenation.

9 Conclusion

This assignment efficiently models a comprehensive implementation for robust modern ASIC pipeline validation. An IEEE-scalable 64:6 lookahead pipelined priority encoder

has passed functional checks completely and been integrated into a 90 nm fabrication flow with complete DFT rule compliance.
Cadence Genus smoothly substituted all 72 structural D-type flip-flops with muxed-scan equivalents, assembling a single 72-bit scan chain with **0 DFT violations** — achieving 100% scannability with minimal structural overhead (72 additional MUX2 cells, +72 routing nets).
Cadence Modus compiled highly efficient pattern vectors — **79 total patterns** — resulting in a perfect **100% Static Fault Coverage** across 2,004 stuck-at fault node definitions. This flawlessly integrates the full DFT engineering pipeline from RTL scan-port declaration, through Genus DFT insertion, to Modus ATPG and committed test vector export.
The complementary PLI-based fault injection in QuestaSim further validated the **observability** of critical internal datapath nodes — including `block_valid`, `row_index[0]`, and `block_index[1]` — confirming 100% detection of all injected faults. The complete testing and verification objective has been verified stringently and successfully.

Final Verification Summary

Functional Verification:	150 tests, 0 mismatches
ATPG Fault Coverage:	100.00% ATCov
PLI Fault Injection:	100.000000% detected
DFT Violations:	0
Scan Chain Status:	Verified (72 FFs, 1 chain)

References

[1] L.-T. Wang, C.-W. Wu, and X. Wen, *VLSI Test Principles and Architectures: Design for Testability*. Morgan Kaufmann, 2006.

[2] M. L. Bushnell and V. D. Agrawal, *Essentials of Electronic Testing for Digital, Memory and Mixed-Signal VLSI Circuits*. Springer Science & Business Media, 2000.

[3] Cadence Design Systems, Inc., *Genus Synthesis Solution User Guide*, version 20.11.

[4] Cadence Design Systems, Inc., *Modus Test Solution User Guide*, version 20.12.

[5] M. Abramovici, M. A. Breuer, and A. D. Friedman, *Digital Systems Testing and Testable Design*. IEEE Press, 1990.

[6] IEEE Std. 1364-2001 (Verilog HDL), *IEEE Standard for Verilog Hardware Description Language*. IEEE, 2001.

A Appendix A: RTL Design Source Code

File: `pe64_top.v` (complete module listing)

```
`timescale 1ns/1ps
module pe4 (
    input  wire [3:0] d,
    output wire [1:0] q,
    output wire       v
);
```

```
    assign q[1] = d[3] | d[2];
    assign q[0] = d[3] | (d[1] & ~d[2]);
    assign v    = |d;
endmodule
```

Listing 1: pe4 – 4:2 base priority encoder.

```
module pe16 (
    input  wire [15:0] d,
    output wire [3:0]  q,
    output wire        v
);
    wire [3:0] row_status;
    reg  [3:0] selected_row;
    wire [1:0] row_index, col_index;
    wire       row_valid;

    assign row_status[0] = |d[3:0];   assign row_status[1] = |d[7:4];
    assign row_status[2] = |d[11:8];  assign row_status[3] = |d[15:12];

    pe4 row_pe (.d(row_status), .q(row_index), .v(row_valid));

    always @(*) begin
        case (row_index)
            2'b00: selected_row = d[3:0];   2'b01: selected_row = d[7:4];
            2'b10: selected_row = d[11:8];  2'b11: selected_row = d[15:12];
            default: selected_row = 4'b0;
        endcase
    end

    pe4 col_pe (.d(selected_row), .q(col_index), .v());
    assign q = {row_index, col_index};
    assign v = row_valid;
endmodule
```

Listing 2: pe16 – 16:4 hierarchical priority encoder.

```
module pe64_lookahead (
    input  wire [63:0] d,
    output wire [5:0]  q,
    output wire        v
);
    wire [15:0] dor;
    wire [3:0]  row_index;
    wire [1:0]  col_index;
    wire        row_valid;
    reg  [3:0]  column_data_reg;

    genvar i;
    generate
        for (i = 0; i < 16; i = i + 1) begin : row_or_logic
            assign dor[i] = |d[(i*4)+3 : i*4];
        end
    endgenerate

```

```
    pe16 row_encoder (.d(dor), .q(row_index), .v(row_valid));

    always @(*) begin
        casex (dor)
            16'b1xxxxxxxxxxxxxxx: column_data_reg = d[63:60];
            16'b01xxxxxxxxxxxxxx: column_data_reg = d[59:56];
            16'b001xxxxxxxxxxxxx: column_data_reg = d[55:52];
            16'b0001xxxxxxxxxxxx: column_data_reg = d[51:48];
            16'b00001xxxxxxxxxxx: column_data_reg = d[47:44];
            16'b000001xxxxxxxxxx: column_data_reg = d[43:40];
            16'b0000001xxxxxxxxx: column_data_reg = d[39:36];
            16'b00000001xxxxxxxx: column_data_reg = d[35:32];
            16'b000000001xxxxxxx: column_data_reg = d[31:28];
            16'b0000000001xxxxxx: column_data_reg = d[27:24];
            16'b00000000001xxxxx: column_data_reg = d[23:20];
            16'b000000000001xxxx: column_data_reg = d[19:16];
            16'b0000000000001xxx: column_data_reg = d[15:12];
            16'b00000000000001xx: column_data_reg = d[11:8];
            16'b000000000000001x: column_data_reg = d[7:4];
            16'b0000000000000001: column_data_reg = d[3:0];
            default:              column_data_reg = 4'b0000;
        endcase
    end

    pe4 col_encoder (.d(column_data_reg), .q(col_index), .v());
    assign q = {row_index, col_index};
    assign v = row_valid;
endmodule
```

Listing 3: pe64_lookahead – 64:6 lookahead priority encoder.

```
module pe64_top (
    input  wire        clk,
    input  wire        rst_n,     // Active-low synchronous reset
    input  wire        enable,
    input  wire [63:0] d,
    output reg  [5:0]  q,
    output reg         v,
    input  wire        scan_en,   // DFT scan enable (active high)
    input  wire        scan_in,   // DFT scan serial input
    output wire        scan_out   // DFT scan serial output (driven
        by Genus)
);
    // Stage 1: Input pipeline registers (64+1 = 65 FFs)
    reg [63:0] d_s1;
    reg        en_s1;

    always @(posedge clk) begin
        if (!rst_n) begin d_s1 <= 64'b0; en_s1 <= 1'b0; end
        else        begin d_s1 <= d;      en_s1 <= enable; end
    end

    // Stage 2: Combinational lookahead encoder (pure logic, no FFs)
    wire [5:0] q_comb;
    wire       v_comb;
    pe64_lookahead u_pe64 (.d(d_s1), .q(q_comb), .v(v_comb));

    // Stage 3: Output pipeline registers (6+1 = 7 FFs)
```

```
    always @(posedge clk) begin
        if (!rst_n)      begin q <= 6'b0; v <= 1'b0; end
        else if (en_s1) begin q <= q_comb; v <= v_comb; end
        else             begin v <= 1'b0; end
    end
    // scan_out left undriven -- Genus connects it post-DFT insertion
endmodule
```

Listing 4: pe64_top – pipelined DFT wrapper (top-level DUT).

B Appendix B: Functional Simulation Testbench

File: `pe64_top_tb.v` (selected excerpt — apply_test task and Section C)

```
task apply_test;
    input [63:0]   in_d;
    input [5:0]    exp_q;
    input          exp_v;
    input [7:0]    test_id;
    reg   [63:0]   captured_d;
    begin
        captured_d = in_d;
        @(negedge clk);
        d      = captured_d;
        enable = 1'b1;
        @(posedge clk);   // stage 1: d_s1 <= d, en_s1 <= 1
        @(posedge clk);   // stage 3: q <= q_comb, v <= v_comb
        #1;               // settle registered outputs

        if (q === exp_q && v === exp_v) begin
            $display("[TEST %0d] PASS | d=64'h%016h => q=%2d, v=%b",
                     test_id, captured_d, q, v);
            pass_count = pass_count + 1;
        end else begin
            $display("[TEST %0d] FAIL | d=64'h%016h => q=%2d (exp %2d
                ), v=%b (exp %b)",
                     test_id, captured_d, q, exp_q, v, exp_v);
            fail_count = fail_count + 1;
        end

        @(negedge clk); enable = 1'b0;
        @(posedge clk); @(posedge clk); // flush pipeline
    end
endtask
```

Listing 5: apply_test task – pipeline-aware stimulus driver.

```
// Section C: Walking-1 (all 64 bit positions)
$display("\n--- Section C: Walking '1' Test (bit 0 to bit 63) ---");

for (i = 0; i < 64; i = i + 1) begin
    walk_vec = (64'h0000000000000001 << i);

    @(negedge clk); d = walk_vec; enable = 1'b1;
    @(posedge clk); @(posedge clk); #1;

    if (q === i[5:0] && v === 1'b1)
```

```
        pass_count = pass_count + 1;
    else begin
        $display("[WALK-1 bit %2d] FAIL | q=%2d (exp %2d), v=%b",
                 i, q, i[5:0], v);
        fail_count = fail_count + 1;
    end

    @(negedge clk); enable = 1'b0;
    @(posedge clk); @(posedge clk);
end
```

Listing 6: Section C – Walking-1 test (64 bit positions).

C Appendix C: DFT and Synthesis Scripts (Genus)

File: run_genus_dft.tcl

```
# ==========================================================================
# Genus Synthesis + DFT Insertion Script
# Design: pe64_top (64:6 Scalable Pipelined Priority Encoder)
# Tool: Cadence Genus Synthesis Solution 20.1
# Library: 90nm (slow.lib)
# ==========================================================================
set_db init_lib_search_path /home/install/FOUNDRY/digital/90nm/dig/lib/
set_db library slow.lib

# Step 1: Read RTL
read_hdl ./pe64_top.v
elaborate pe64_top
check_design -unresolved

# Step 2: Apply timing constraints
read_sdc ./pe64_top.sdc

# Step 3: Pre-synthesis DFT definitions
set_db dft_scan_style muxed_scan
set_db dft_prefix DFT_
define_dft shift_enable -name scan_en_sig -active high scan_en
define_dft test_clock   -name clk_test -period 10000 clk

# Step 4: Synthesis
set_db syn_generic_effort high;  syn_generic
set_db syn_map_effort high;      syn_map
set_db syn_opt_effort high;      syn_opt

# Step 5: Pre-DFT reports
report_timing > ./pre_dft_timing.rpt
report_area   > ./pre_dft_area.rpt
report_power  > ./pre_dft_power.rpt
write_hdl     > ./pe64_top_pre_dft.v

# Step 6: DFT insertion
```

```
check_dft_rules > ./dft_rules_check.rpt
replace_scan
define_scan_chain -name chain1 -sdi scan_in -sdo scan_out \
                  -non_shared_output
connect_scan_chains
syn_opt -incr

# Step 7: Post-DFT reports and export
report_timing   > ./post_dft_timing.rpt
report_area     > ./post_dft_area.rpt
report_power    > ./post_dft_power.rpt
report_dft_chains > ./scan_chains.rpt

write_hdl     > ./pe64_top_post_dft.v
write_sdf     > ./pe64_top_post_dft.sdf
write_sdc     > ./pe64_top_post_dft.sdc
write_scandef > ./pe64_top.scandef
write_dft_atpg -library ./pe64_top_post_dft.v -directory ./
gui_show
```

Listing 7: Genus synthesis and DFT insertion script.

File: ***pe64_top.sdc*** *— Timing constraints*

```
# Primary clock (100 MHz)
create_clock -name clk -period 10.0 [get_ports clk]

# Clock quality
set_clock_transition    0.10 [all_clocks]
set_clock_uncertainty -setup 0.30 [all_clocks]
set_clock_uncertainty -hold  0.10 [all_clocks]

# I/O delays
set_input_delay  2.0 -clock clk [get_ports {d[*] enable rst_n}]
set_input_delay  0.0 -clock clk [get_ports {scan_in scan_en}]
set_output_delay 2.0 -clock clk [get_ports {q[*] v scan_out}]

# False paths for DFT ports
set_false_path -from [get_ports scan_en]
set_false_path -from [get_ports scan_in]
set_false_path -to   [get_ports scan_out]

set_operating_conditions -library slow slow
```

Listing 8: SDC timing constraints for pe64_top.

D Appendix D: ATPG Generation Script (Modus)

File: ***runmodus.atpg.tcl***

```
set_db workdir ./
set WORKDIR [get_db workdir]
set_option stdout summary
set ::env(CDS_LIC_REPORT) yes
source $::env(Install_Dir)/bin/64bit/test_checks.tcl
file delete -force $WORKDIR/tbdata
file delete -force $WORKDIR/testresults

```

```
# Locate 90nm library models
set LIBRARY ""
catch {
    set raw_files [exec bash -c "find /home/install/FOUNDRY/digital
       /90nm/dig/ \
        -type f | grep -E \"\\.v\$|\\.v\\.gz\$\""]
    set LIBRARY [split $raw_files "\n"]
}

# Build model
build_model \
    -cell pe64_top \
    -techlib $LIBRARY \
    -designsource $WORKDIR/pe64_top.test_netlist.v \
    -allowmissingmodules yes

# Build & verify FULLSCAN test mode
build_testmode -testmode FULLSCAN \
    -assignfile $WORKDIR/pe64_top.FULLSCAN.pinassign \
    -modedef FULLSCAN
report_test_structures -testmode FULLSCAN -reportscanchain all
verify_test_structures -testmode FULLSCAN

# Stuck-at fault model (no dynamic faults)
build_faultmodel -includedynamic no

# ATPG pattern generation
create_logic_tests -experiment pe64_top_atpg -testmode FULLSCAN

# Switching analysis
write_toggle_gram -experiment pe64_top_atpg -testmode FULLSCAN

# Export Verilog test vectors (parallel scan format)
write_vectors -inexperiment pe64_top_atpg \
              -testmode FULLSCAN \
              -language verilog \
              -scanformat parallel

# Commit patterns to master database
commit_tests -inexperiment pe64_top_atpg -testmode FULLSCAN

puts ">>> ATPG Run Complete!"
display_schematic
```

Listing 9: Modus ATPG execution script for pe64_top.

E Appendix E: Selected Verification Logs

E.1 Scan Chain Structural Report

File: derived from `report_test_structures -reportscanchain all`

```
Reporting 1 scan chain (muxed_scan)
Chain 1: chain1
  scan_in:        scan_in
  scan_out:       scan_out
  shift_enable:   scan_en (active high)
  clock_domain:   clk_test (edge: rise)
```

```
  length:        72

  Position  Type       Block Name
  --------  ---------  ----------------------------------
  bit   1   rDFF_cS    inst:\d_s1_reg[0]
  bit   2   rDFF_cS    inst:\d_s1_reg[1]
  ...
  bit  64   rDFF_cS    inst:\d_s1_reg[63]
  bit  65   rDFF_cS    inst:\en_s1_reg
  bit  66   rDFF_cS    inst:\q_reg[0]
  bit  67   rDFF_cS    inst:\q_reg[1]
  bit  68   rDFF_cS    inst:\q_reg[2]
  bit  69   rDFF_cS    inst:\q_reg[3]
  bit  70   rDFF_cS    inst:\q_reg[4]
  bit  71   rDFF_cS    inst:\q_reg[5]
  bit  72   rDFF_cS    inst:\v_reg
  ------------------------------------------------

INFO (TSV-068): Length of longest scan chain is 72 bit positions,
                which is 100% of average chain length 72
                (1 valid scan chain, 72 total bits).
INFO (TSV-378): Scan chain (scan_in -> scan_out) is controllable
                and observable. Length: 72 bit positions.
INFO (TSV-567): 1 controllable scan chain(s) fed by Scan-In (SI).
INFO (TSV-568): 1 observable scan chain(s) feeding Scan-Out (SO).
INFO (TSV-908): verify_test_structures processing complete.
```

E.2 Modus Design Structural Summary

```
INFO (TLM-055): Design Summary
  Hierarchical Model:          Flattened Model:
    2,223 Blocks                 1,132 Blocks
    6,956 Pins                   1,132 Nodes
    3,879 Nets

  Primary Inputs:              Primary Outputs:
    69 Input Only                8 Output Only
    69 Total Inputs              8 Total Outputs

  Tied Nets: 0                 Dotted Nets: 0

  72  MUX2s                (scan mux cells)
  72  Rising Edge Flop w/Set-Dominant and Reset Port
  72  Total Flops

  203 Technology Library Cell Instances
```

E.3 DFT Execution Setup Profile

```
Design Name:   pe64_top
Scan Style:    muxed_scan

Test Signal Objects:
  shift_enable:
    object name: scan_en_sig
    pin name:    scan_en
    active:      high
```

```
    user defined: true

Test Clock Objects:
  test_clock:
    object name:  clk_test
    source:       clk
    period:       10000.0 ps
    hookup_pin:   clk

DFT Actual Scan Chains:
  chain1:
    scan-in:    scan_in
    scan-out:   scan_out
    shift_enable: scan_en
    length:     72
    test_clock domain: clk_test
    test_clock edge:   rise
```

E.4 Commit Tests Final Statistics

```
Testmode and Global statistics after commit_tests
              Testmode  Global   Total   Tested  Possibly  Redund
                 Untested
Total Static   100.00  100.00    2004     2004       0        0
          0
Total Dynamic                       0

INFO (TBD-831): commit_tests completed successfully.

Maximum Memory =  6,312,496 bytes
CPU Time       =  0:00:00.09
Elapsed Time   =  0:00:02.24
Date Ended:    Thursday Apr 16 13:22:51 2026 IST
```